

Unit - 2

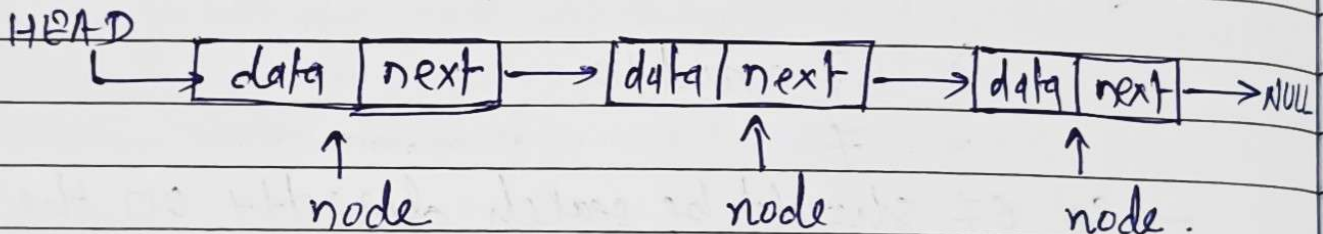
Linked List :

A linked list is a linear data structure, in which the elements are not stored at contiguous memory locations.

The elements in a linked list are linked using pointers.

A linked list consists of nodes where each node contains a data field and a reference link to the next node in the list.

• Representation



It is basically chains of nodes, each node contains information such as data and a pointer to the next node in the chain.

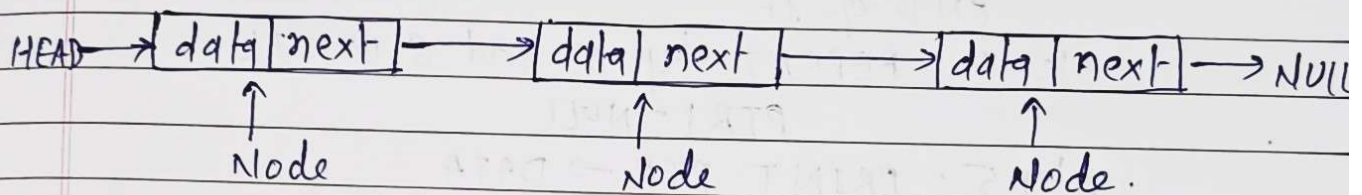
There is a head pointer, which points to the first element of the linked list.

If the list is empty then it simply points to null or nothing.

• Types of Linked List

1. Single-linked list
2. Double linked list
3. Circular linked list

★ 1. Single - Linked List



Singly Linked List

- most common type of linked list.
- Each node has data and an address field that contains a reference to the next node.
- Traversing:
 - A linked list will be traversed from the head node to the last node of the list.
 - Traversing means visiting every node of the linked list to perform some operation.

* • Traversing a Single linked list

Algo

step 1: set PTR = HEAD

step 2: IF PTR = NULL

step 3: WRITE "EMPTY LIST"

GOTO step 7

END of IF

step 4: REPEAT step 5 and 6 UNTIL
PTR = NULL

step 5: PRINT PTR → DATA

step 6: PTR = PTR → NEXT

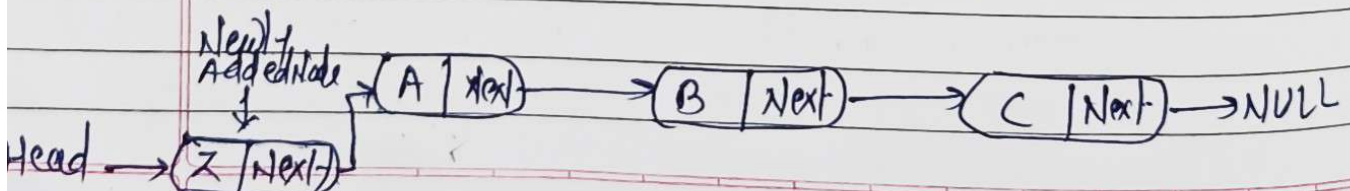
step 7: EXIT.

* • Insertion into a single linked list

- You can add elements to either the beginning, middle or end of the linked list.

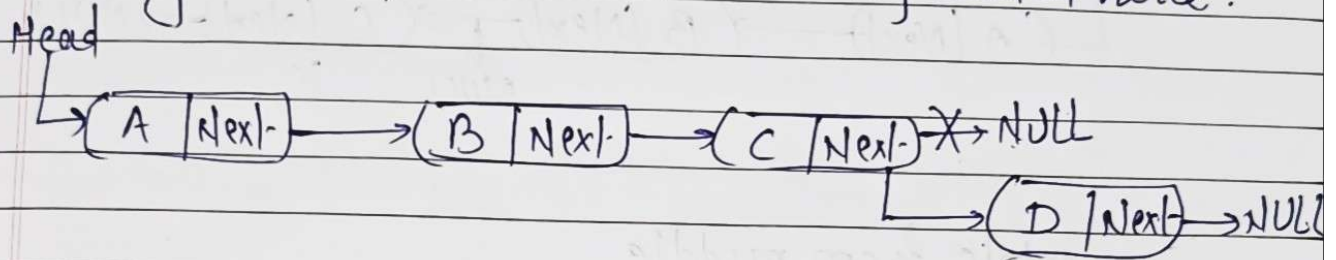
1. Insert-at-the beginning

- Allocate memory for new node.
- store data
- change next of new node to point to head
- change head to point to recently created node.



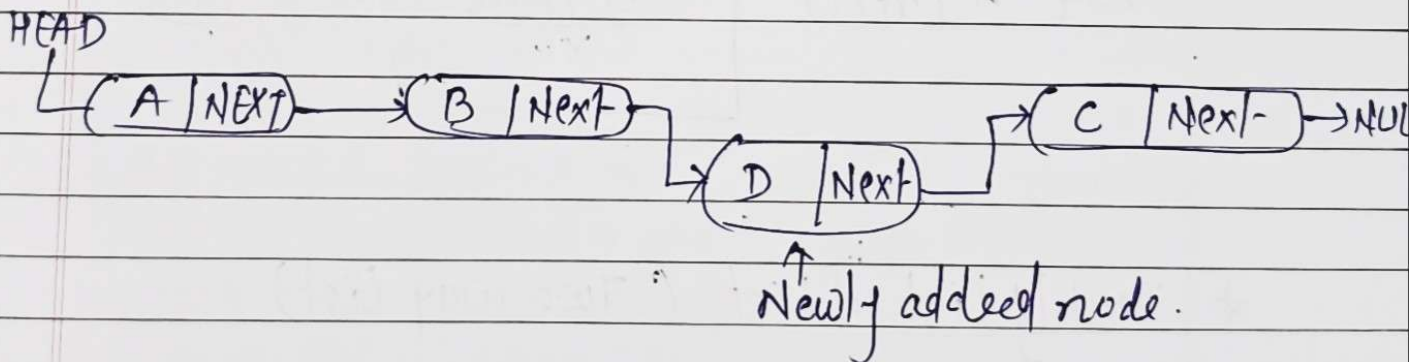
2. Insert at the end

- Allocate memory for new node.
- store data
- Traverse to last node
- change next of last node to recently created node.



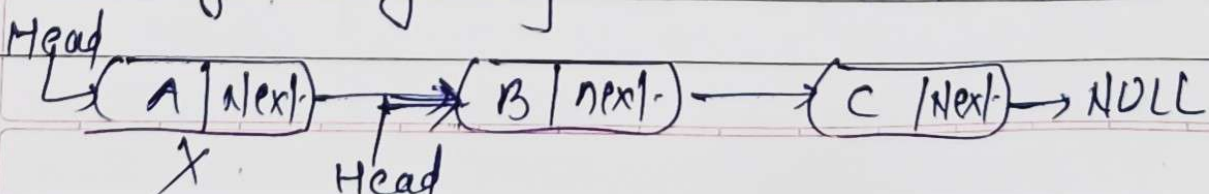
3. Insert at the middle

- Allocate memory and store data for new node.
- Traverse to node just before the required position of new node.
- change next pointers to include new node in between.

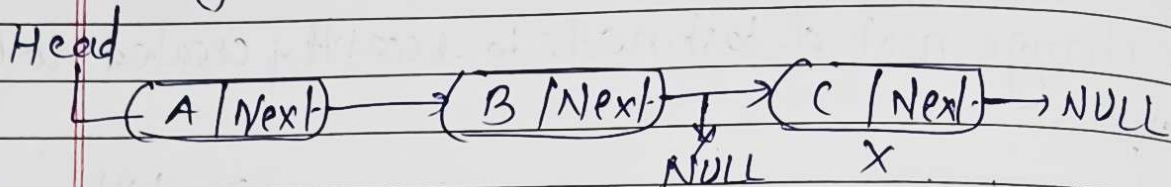


★ Deletion into a Single linked list

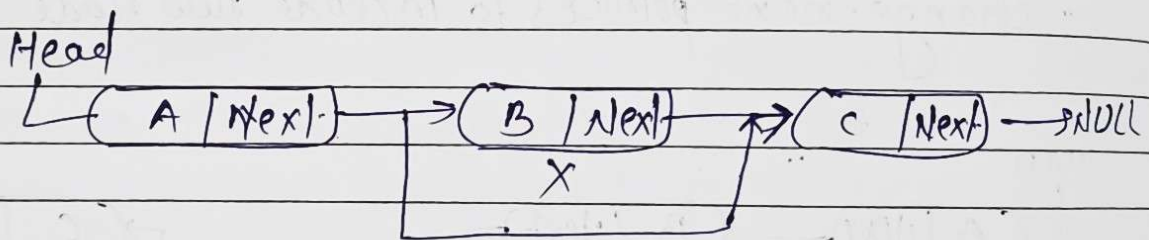
1. Delete from beginning → Point head to the second node



- Delete from End
- Traverse to second last element
- change its next pointer to null.



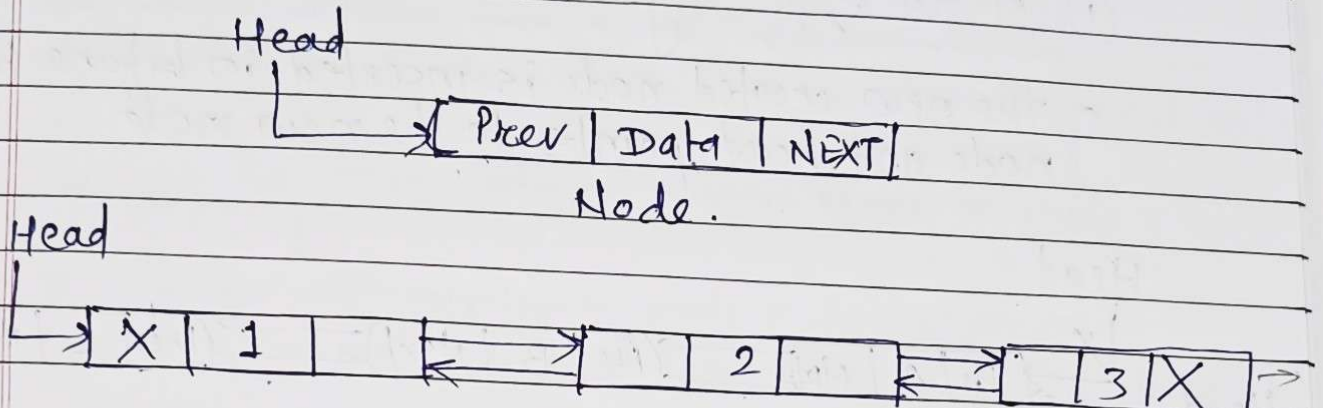
- Delete from middle
- Traverse to element before the element to be deleted.
- change next pointers to exclude the node from the chain.



★ Doubly Linked List (Two-way List)

- Doubly linked list is a complex type of linked list in which a node contains a pointer to the previous as well as the next node in the sequence.

- A node consists of three parts:
node data, pointer to the next node, pointer to the previous node.



Doubly linked list

- In doubly linked list traversing will be done in both direction. (forward and backward).

Traversing

Step 1: IF HEAD == NULL
WRITE "UNDERFLOW"
GOTO Step 6
[END OF IF]

Step 2: Set PTR = HEAD

Step 3: Repeat Step 4 and 5 while PTR != NULL

Step 4: While PTR → data

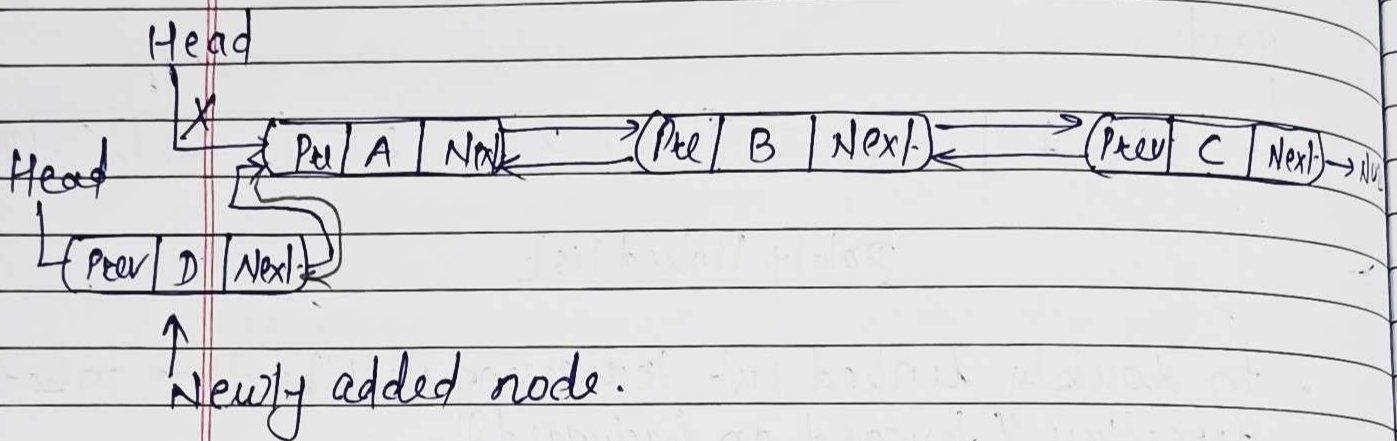
Step 5: PTR = PTR → next

Step 6: Exit.

★ Inserting into Doubly linked list-

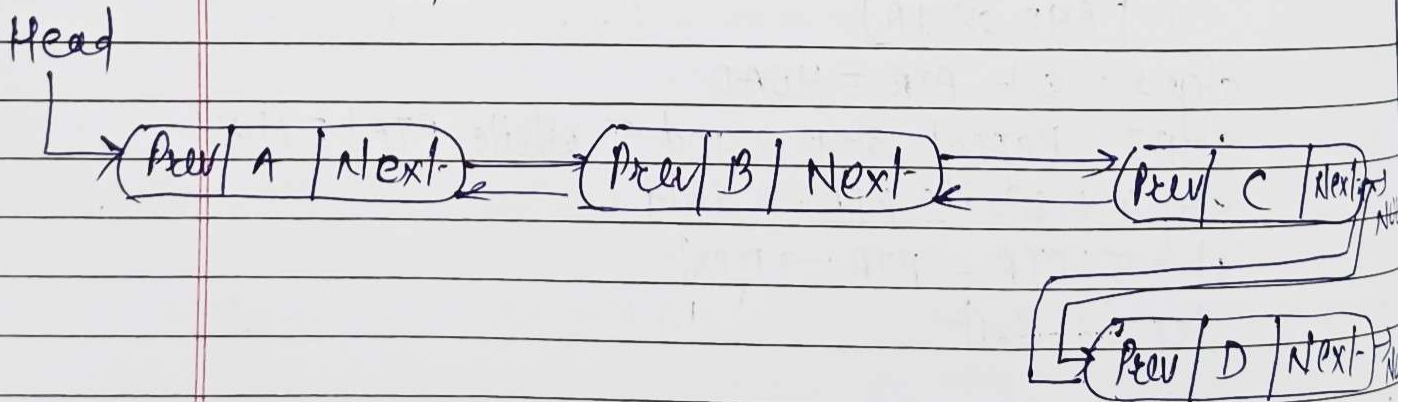
(1) At the beginning

- The new created node is inserted in before the head node and head points to the new node.



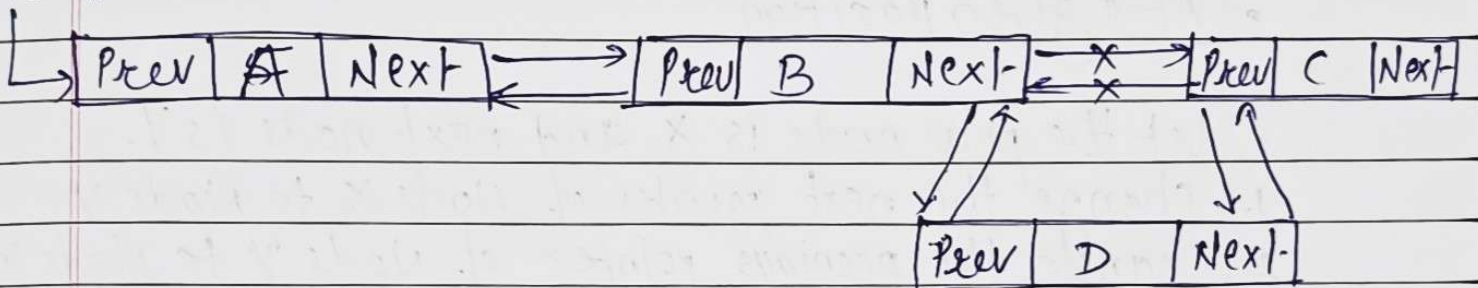
(2) At the end

- The new created node is inserted at the end of the list and tail points to the new node.



- At a given position
 - Reverse the given DLL to that location.
1. change the next pointer of new node to the next pointer of node x.
 2. change the prev pointer of next node of node x to the new node.
 3. change the next pointer of node x to new Node.
 4. change the prev pointer of new Node to the node x.

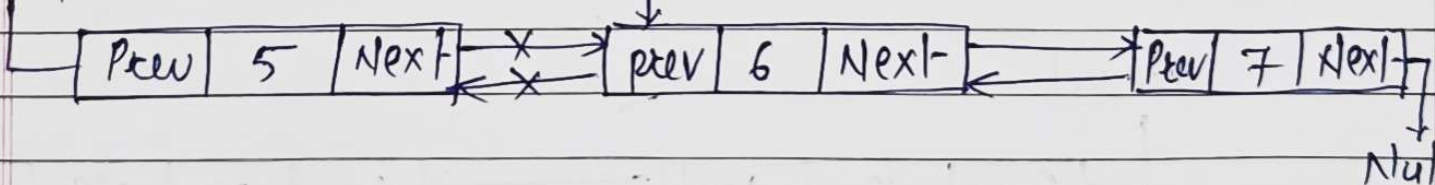
Head



★ Deletion from Doubly linked List

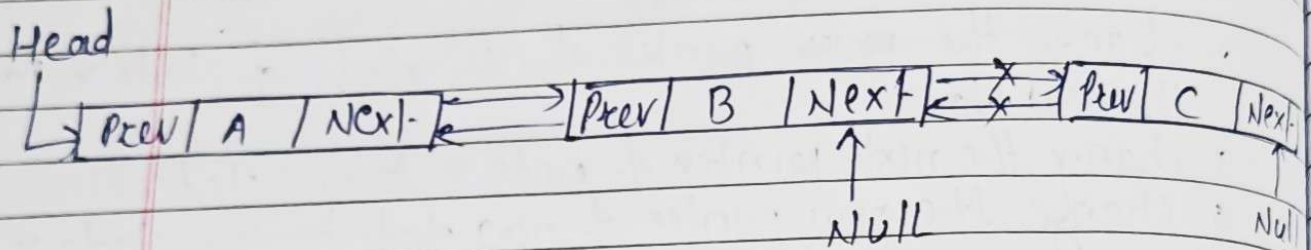
- At the beginning - Move head to the next node to delete the node at the beginning and make previous pointer of current head to NULL.

Head X



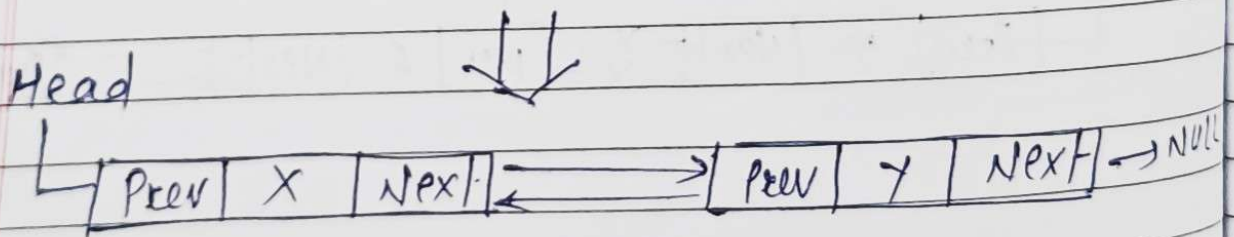
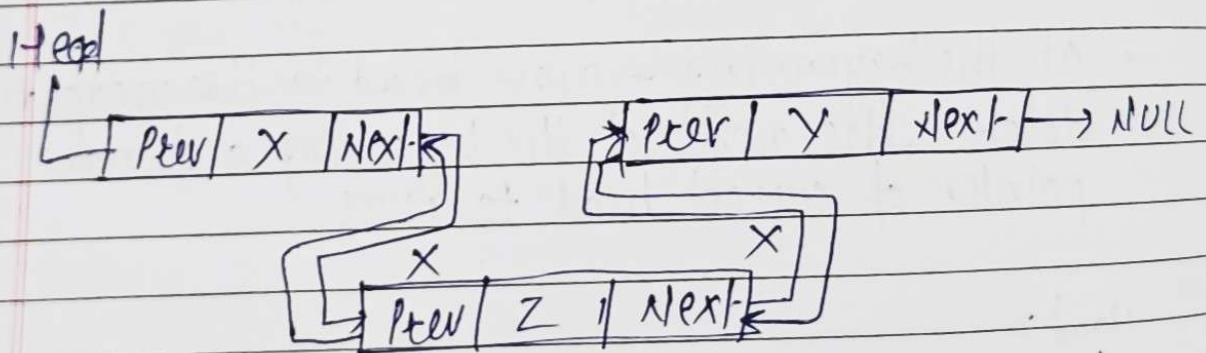
• At the last

- move tail to the previous node to delete the node at the end and make next pointer of tail node to NULL.



• At a given position

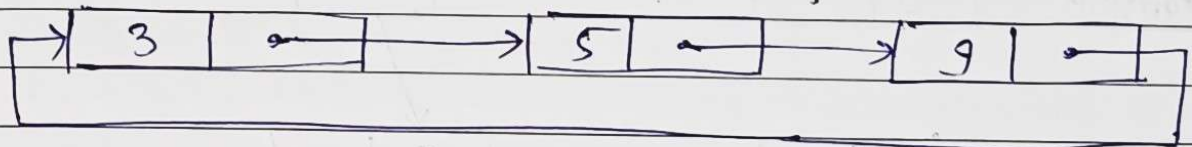
- let the prev node is X and next node is Y.
- 1. change the next pointer of Node X to Node Y.
- 2. change the previous pointer of Node Y to Node X.



★ Circular linked List

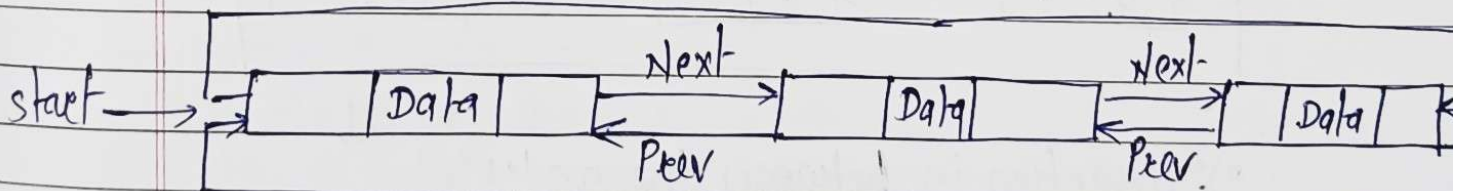
• Circular singly linked list

- The last node of the list contains a pointer to the first node of the list.
- We traverse the list until we reach the same node where we started.
- It has no beginning or end.



• Circular Doubly linked list

- It has properties of both doubly linked list and circular linked list in which two consecutive elements are linked or connected by the previous and next pointer and the last node points to the first node by the next pointer and also the first node points to the last node by the previous pointer.

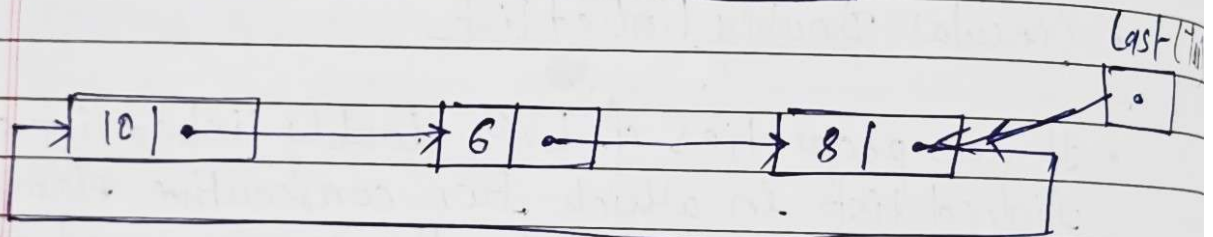
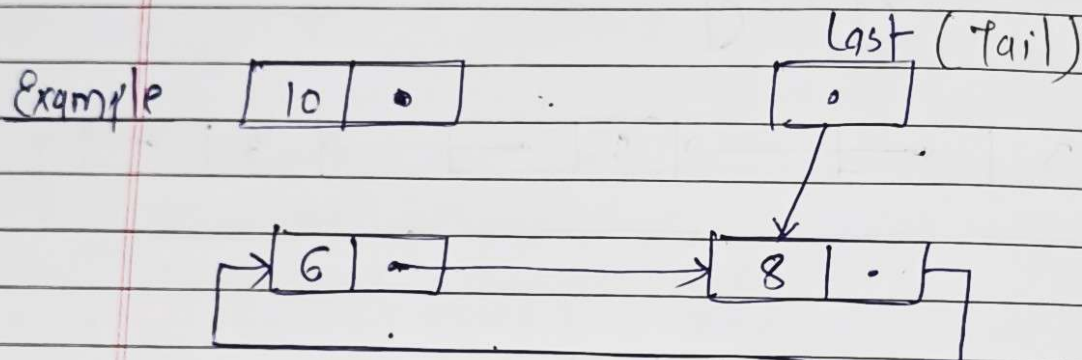


• Operations on the Circular linked list

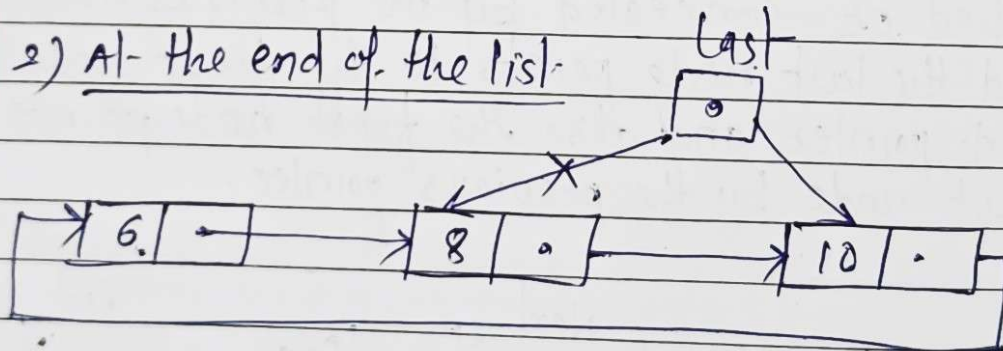
• Insertion

1) Insertion at the beginning of the list:

- Create a node, say T
- make $T \rightarrow \text{next} = \text{last} \rightarrow \text{next}$
- $\text{last} \rightarrow \text{next} = T$



2) At the end of the list

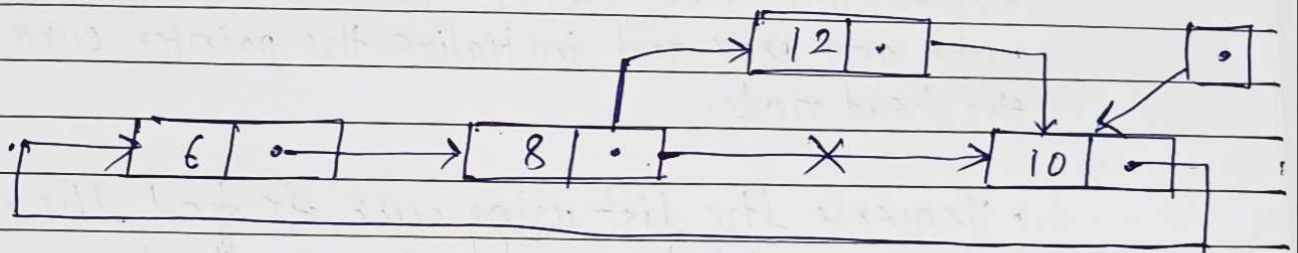


3) Insertion in between the nodes

- Create a node, say T
- make $T \rightarrow \text{next} = \text{last} \rightarrow \text{next}$
- $\text{last} \rightarrow \text{next} = T$
- $\text{last} = T$

3) Insertion in between the nodes.

- Create a node, say T
- Search for the node after which T needs to be inserted say that node is P.
- Make $T \rightarrow \text{next} = P \rightarrow \text{next}$
- $P \rightarrow \text{next} = T$



• Deletion

1) Delete the node if it is the only node in the circular linked list.

→ Free the node's memory

→ The last value should be null, A node always points to another node, so NULL assignment is not necessary.

2) Deletion of the last node

- Locate the node before the last node.
- Keep the address of the node next to the last node in temp
- Delete the last memory.
- Put temp at the end.

3) Delete any node from the circular linked list:

Algorithm

Case 1: List is empty.

- If the list is empty we will simply return.

Case 2: List is not empty.

- If the list is not empty then we define two pointers `curr` and `prev` and initialize the pointer `curr` with the head node.

- Traverse the list using `curr` to find the node to be deleted and before moving to `curr` to the next node, every time set `prev = curr`.

- If the node is found, check if it is the only node in the list. If yes, set `head = NULL` and `free(curr)`.

- If the list has more than one node, check if it is first node of the list. Condition to check this (`curr == head`). If yes, then move `prev` until it reaches the last node.

After `prev` reaches the last node, set `head = head → next` and `prev → next = head`. Delete `curr`.

- If `curr` is not the first node, we check if it is the last node in the list. Condition to check this is (`curr → next == head`)

- If `curr` is the last node. set `prev → next = head` and delete the node `curr` by `free`.

- If the node to be deleted is neither the first node nor the last node, then set $\text{prev} \rightarrow \text{next} = \text{curr} \rightarrow \text{next}$ and delete curr.
- If the node is not present in the list return head and don't do anything.

Time Complexity: $O(N)$
Auxiliary Space $O(1)$

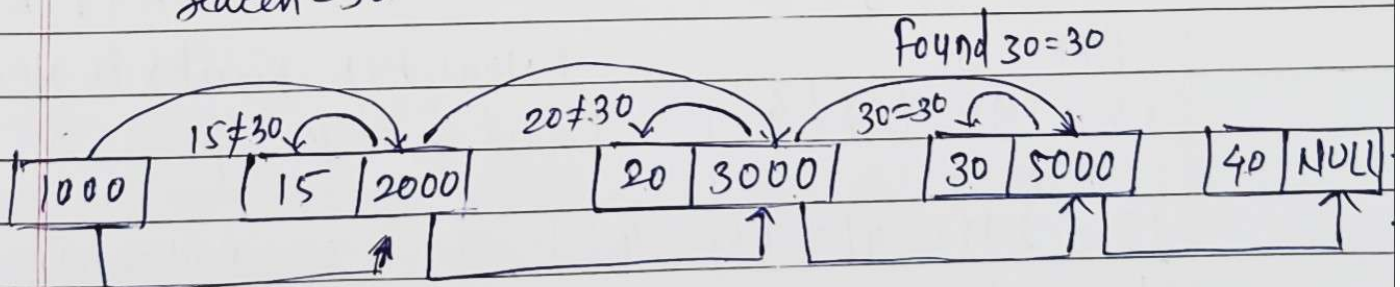
* searching a linked list

1) Iterative Approach

- Initialize a node pointer
 $\text{current} = \text{head}$
- Do the following while current is not null.
 - If the current value (i.e., $\text{current} \rightarrow \text{key}$) is equal to the key being searched return true.
 - otherwise, move to the next node.
($\text{current} = \text{current} \rightarrow \text{next}$)
- If the key is not found, return false.

Time Complexity : $O(N)$ - N = Number of nodes
Auxiliary Space : $O(1)$

Search = 30



2) Recursive Approach

Follow the below steps to solve the problem.

- If the head is NULL, return false.
- If the head's key is the same as X, return true.
- Else recursively search in the next node.

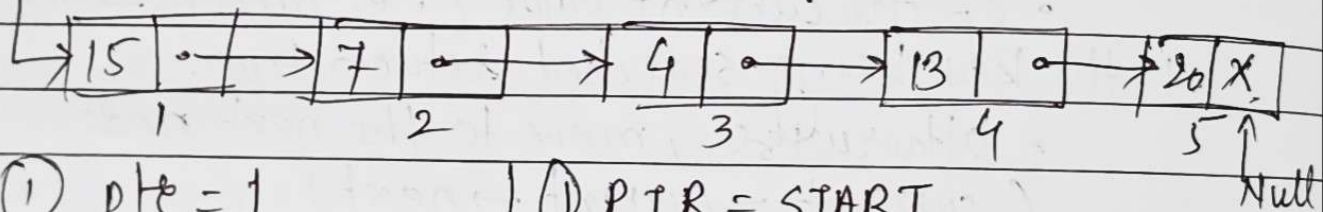
Time Complexity : $O(N)$

Auxiliary Space : $O(N)$

★ List is Unsorted

- Algo - Search(info, link, start, item, loc)

start



Search
ITEM=13

① PTR = 1

② Repeat- PTR $\neq$ NULL
 $13 = \text{INFO}[1] \neq 15$

PTR = 2

$13 = \text{INFO}[2] \neq 7$

PTR = 3

$13 = \text{INFO}[3] \neq 4$

PTR = 4

$13 = \text{INFO}[4] = 13$

LOC = 4

① PTR = START

② Repeat- PTR $\neq$ NULL

③ If- ITEM = INFO(PTR)
then,

Set- LOC = PTR & Exit
Else.

Set- PTR = LINK[PTR]

[Now PTR points to Next Node]

[End of step 2.]

④ [Search is Unsuccessful]

LOC = NULL

⑤ Exit.

* list is Sorted

Search (INFO, LINK, START, ITEM, LOC)

Algo

- ① Set PTR = START
- ② Repeat step (3) while PTR $\neq$ NULL
- ③ If $ITEM < INFO[PTR]$
Set PTR = LINK [PTR]
- Else If $ITEM = INFO[PTR]$ then
Set LOC = PTR, and Exit.
- ④ Else
Set LOC = NULL & Exit

Start



① ITEM = 13

③ PTR = 1

$13 > INFO[1] = 4$

PTR = LINK[1] = 2

INFO

$13 > INFO[2] = 9$

PTR = LINK[2] = 3

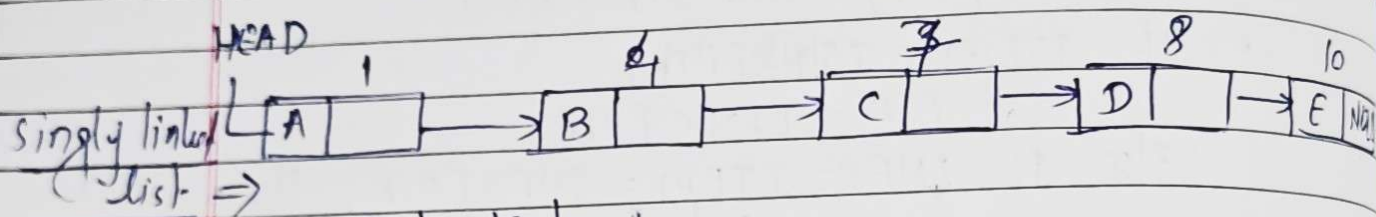
$13 > INFO[3] = 13$

$13 = 13$

LOC = 3 Exit.

• Memory Allocation

- linked list does the dynamic memory allocation.
- memory is divided into a number of blocks of similar size when allocating a linked list.
- linked list is implemented on the heap memory rather than the stack memory.



Data Next -

start.	1	A	4
	2		
	3		
	4	B	7
	5		
	6		
	7	C	8
	8	D	10
	9		
	10	E	-1 OR NULL

• Here start = 1, to the first data is stored at address 1, where is A.

• the corresponding 'Next' stores the address of the next node which is 4.

So next data item is at 4.

-1 OR NULL $\Rightarrow$ Denotes end of the linked list.

• Garbage collection

- It is a dynamic approach to automatic memory management and heap allocation that processes and identifies dead memory blocks and reallocates storage for reuse.

- The primary purpose of garbage collection is to reduce memory leaks.
- The basic principles of garbage collection are finding data objects in a program that can not be accessed in the future and reclaiming the resources used by those objects.
- Garbage Collection usually takes place in two steps.
- First - The program runs through all lists, tagging those cells which are currently in use and then the computer runs through the memory, collecting all untagged space of - onto the free storage list.
- There are two conditions in garbage collection.
 - ① Overflow
 - Sometimes new data has to be inserted into a data structure but there is no available space.
 - That means free storage list is empty.
 - This situation is called overflow.

② Underflow

- The term underflow refers to the situation where to delete data from a data structure that is empty.
- The overflow occurs in the list when $AVAIL = NULL$
- The underflow occurs in the list when $START = NULL$